

CP — Comparador de Preços

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	CP — Comparador de Preços
Gerente de Projeto	Marcio Moreira Kuskowski
Gerente/Responsável de GCS/DevOps	Marcio Moreira Kuskowski
Fábrica de Software / Squad	Individual — Marcio Moreira Kuskowski (Engenharia de Software, Católica SC)
Repositório principal	A ser criado no GitHub (repositório ainda não publicado — ver seção 4.3)

Histórico de Revisões

Data	Versão	Descrição	Autor
22/09/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD), a partir do template institucional.	Marcio Moreira Kuskowski

Sumário

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do CP — Comparador de Preços, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD) e automação de pipelines. Por se tratar de um projeto individual, em estágio inicial de desenvolvimento, as atividades de GCS são desenhadas com o mínimo de burocracia possível e o máximo de automação viável para um único desenvolvedor.

1.1 Finalidade

Estabelecer um padrão a ser seguido pelo responsável pelo projeto, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação, à medida que o repositório e a automação forem sendo estruturados.

1.2 Escopo

Este PGC é destinado a Marcio Moreira Kuskowski, único integrante responsável pelo desenvolvimento e operação do CP — Comparador de Preços, aplicação em Python cujo objetivo é comparar preços de produtos de hardware/tecnologia entre varejistas brasileiros. O escopo cobre o código-fonte da aplicação (versão web e versão desktop em Tkinter), os scripts de coleta/atualização de preços e a documentação técnica associada.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
CCM	Comitê de Controle de Mudanças (não aplicável formalmente — projeto individual)
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que o desenvolvedor integra na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) — ex.: 1.0.0
Scraping/Coleta de Preços	Rotina automatizada de consulta a sites de varejistas para obtenção do preço atual de um produto

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração — Enfoque DevOps (Católica SC / RUP 7.0, IBM — adaptado).
- Repositório e documentação técnica do CP — Comparador de Preços (a serem publicados no GitHub).

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do fluxo de trabalho: cada commit integrado, testado e, futuramente, implantado automaticamente, gera evidências de rastreabilidade (histórico do Git, tags de versão, execuções de pipeline), reduzindo o esforço manual necessário para manter o controle do projeto.

2. Cultura DevOps e Papéis na Gerência de Configuração

Por se tratar de um projeto individual, todos os papéis abaixo são exercidos por Marcio Moreira Kuskowski. A tabela documenta as responsabilidades de cada papel para fins de clareza do processo, ainda que concentradas em uma única pessoa.

Papel	Responsabilidades	Responsável
Gerente de Projeto	Define prioridades, escopo das funcionalidades (comparação de preços, sites suportados) e acompanha o progresso.	Marcio Moreira Kuskowski
Engenheiro DevOps	Configura o repositório Git, define a estratégia de branching/versionamento e estrutura futura automação de CI/CD.	Marcio Moreira Kuskowski
Desenvolvedor	Implementa a aplicação em Python (web e desktop/Tkinter) e os módulos de coleta de preços.	Marcio Moreira Kuskowski
QA / Testes	Valida manualmente as funcionalidades e, futuramente, escreve testes automatizados para os módulos de coleta e comparação.	Marcio Moreira Kuskowski

Nota: por não haver equipe, o fluxo de revisão por pares ("quatro olhos") descrito na seção 5.2 não se aplica neste estágio; ele é registrado no documento como prática recomendada para caso o projeto venha a receber colaboradores.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas adotadas ou planejadas para o projeto:

Categoria	Ferramenta(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento do código-fonte (a ser criado)
Linguagem / Runtime	Python 3	Desenvolvimento da aplicação web e da aplicação desktop (Tkinter)
Orquestração de CI	GitHub Actions (planejado)	Automação de testes e verificação de disponibilidade dos sites monitorados
Gestão de dependências	pip / requirements.txt	Fixação e versionamento das bibliotecas Python utilizadas
Rastreamento de tarefas	GitHub Issues (planejado)	Registro de melhorias, como novos sites suportados

3.2 Ambientes e Infraestrutura

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento (local)	Execução e testes da aplicação web e desktop na máquina do desenvolvedor.	Alterações no código local
Produção (execução local do usuário)	Uso final da aplicação desktop/web pelo próprio usuário, sem servidor remoto dedicado neste estágio.	Release estável publicada no repositório (tag de versão)

Por ora, o projeto não utiliza ambiente de homologação/staging nem infraestrutura provisionada em nuvem, dado o estágio inicial e o caráter individual do desenvolvimento. A evolução para um ambiente de homologação será avaliada caso a aplicação passe a ser hospedada como serviço web.

3.3 Estratégia de Branching e Versionamento

Recomenda-se estratégia de Trunk-Based Development simplificada, adequada a um desenvolvedor único:

- main — branch principal, sempre no estado mais estável possível.
- feature/<descrição-curta> — branches de curta duração para cada nova funcionalidade, como suporte a um novo site de varejista.
- fix/<descrição> — correções pontuais de bugs identificados na coleta ou comparação de preços.

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor), permitindo histórico claro de evolução mesmo sem geração automática de changelog neste estágio.

4. Identificação da Configuração

4.1 Itens de Configuração

- Código-fonte da aplicação web e da aplicação desktop (Tkinter).
- Módulos de coleta/atualização de preços por site de varejista.
- Arquivo de dependências (requirements.txt).
- Definições de pipeline como código, quando implementadas (ex.: .github/workflows).
- Arquivos de configuração (ex.: lista de sites suportados, parâmetros de busca).
- Documentação técnica (README, este PGCS).

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<descrição-curta>	feature/suporte-site-kabum
Commit	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(coleta): adiciona suporte ao site Kabum
Tag de release	Versionamento Semântico	v0.1.0

4.3 Estrutura do Repositório

O repositório ainda não foi criado no GitHub; será publicado por Marcio logo após a formalização deste plano. A estrutura planejada é:

- /web — versão web do comparador de preços
- /desktop — versão desktop em Python/Tkinter
- /scraper — módulos de coleta/atualização de preços por site
- /tests — testes automatizados (planejado)
- /docs — documentação técnica, incluindo este PGCS

5. Integração Contínua (CI)

Para este projeto, a "Integração Contínua" tem um enfoque funcional específico: além da validação de build e testes, o pipeline de CI é responsável por manter os preços atualizados e por ampliar, de forma controlada, a variedade de sites verificados.

5.1 Etapas do Pipeline de CI Planejado

- Checkout do código e instalação de dependências (requirements.txt).
- Execução de testes automatizados dos módulos de coleta de preços (quando implementados).
- Execução agendada (scheduled job) do módulo de atualização de preços, consultando os sites de varejistas configurados.
- Validação de que cada novo site adicionado retorna dados no formato esperado (preço, nome do produto, disponibilidade) antes de ser integrado à branch principal.
- Registro de falhas de coleta (ex.: mudança de layout de um site) como issue para correção.

5.2 Política de Revisão de Código

Como o projeto é individual, não há revisão por pares neste estágio. Ainda assim, recomenda-se que toda alteração relevante — especialmente a adição de um novo site de coleta — passe por uma auto-revisão via Pull Request antes do merge na main, preservando o histórico e permitindo reverter mudanças com facilidade caso um site pare de funcionar.

5.3 Critérios de Qualidade (Quality Gate)

- A aplicação deve iniciar (web e desktop) sem erros antes de qualquer merge na main.
- Todo novo site adicionado ao comparador deve retornar preços válidos em pelo menos uma execução de teste antes de ser disponibilizado.
- Nenhuma credencial ou chave de API deve ser versionada diretamente no repositório.

6. Entrega e Implantação Contínua (CD)

Como o CP — Comparador de Preços ainda não é hospedado como serviço remoto, a "implantação" corresponde, neste estágio, à publicação de uma nova versão estável (tag) no repositório para uso local pelo usuário. O pipeline de CD será expandido caso a versão web venha a ser hospedada em um servidor ou serviço de nuvem.

6.1 Etapas do Pipeline de CD Planejado

- Geração de uma tag de versão (SemVer) na main após validação da nova funcionalidade ou correção.

- Atualização do README com as novas funcionalidades/sites suportados.
- Disponibilização da versão atualizada para download/uso via GitHub Releases.
- (Futuro) Deploy automático da versão web em serviço de hospedagem, caso o projeto evolua para esse modelo.

6.2 Estratégias de Implantação

Neste estágio, aplica-se uma implantação direta (substituição simples da versão anterior pela nova release). Estratégias mais avançadas — como Blue-Green ou Canary — serão avaliadas apenas se o projeto passar a operar como serviço web hospedado com múltiplos usuários simultâneos.

6.3 Rollback e Recuperação

Toda alteração é reversível através do histórico do Git: em caso de falha (ex.: um site de coleta quebrar a aplicação), a branch main pode ser revertida para a última tag estável conhecida.

7. Controle de Configuração e Mudança

7.1 Processo de Solicitação de Mudança

Mudanças de rotina (novas funcionalidades, correções, novos sites de coleta) seguem o fluxo: abertura de issue → branch de feature → auto-revisão → merge na main. Esse fluxo constitui o registro formal e rastreável da mudança, mesmo sem um Comitê de Controle de Mudanças formal.

7.2 Comitê de Controle de Mudança (CCM)

Não se aplica formalmente a este projeto individual. Decisões de maior impacto (ex.: migrar a coleta de preços para uma API paga, ou hospedar a versão web) são registradas no README e no histórico de commits como decisões de arquitetura, cumprindo informalmente o papel do CCM.

8. Baselines e Releases

Cada versão estável publicada gera uma baseline: uma tag SemVer associada ao commit correspondente na main, documentando o estado do código, dos sites suportados e das dependências naquele momento.

8.1 Notas de Versão

As notas de release resumem, por versão, os sites de varejistas adicionados/removidos e as correções realizadas, registradas manualmente no GitHub Releases até que a geração automática a partir de Conventional Commits seja implementada.

9. Estimativa do Status de Configuração

9.1 Monitoramento

O status do projeto é acompanhado através do histórico de commits, das issues abertas (novos sites a suportar, bugs de coleta) e de execuções manuais da aplicação para validar se a atualização de preços continua funcionando corretamente.

9.2 Indicadores de Acompanhamento

Indicador	Descrição
Número de sites suportados	Quantidade de varejistas cujos preços podem ser consultados e comparados
Taxa de sucesso da coleta	Proporção de execuções de atualização de preços concluídas sem erro por site
Frequência de atualização de preços	Intervalo de tempo entre execuções do módulo de atualização

10. Marcos

- Criação e publicação do repositório no GitHub.
- Primeira versão estável (v0.1.0) com suporte a um site de varejista.
- Ampliação para múltiplos sites de varejistas suportados.
- Implementação da rotina automatizada de atualização de preços (CI).
- Avaliação de hospedagem da versão web (evolução do CD).

11. Treinamento e Recursos

Como responsável único pelo projeto, Marcio é o recurso principal de conhecimento técnico, com formação em andamento em Engenharia de Software (Católica SC). Recursos necessários incluem estudo autodirigido sobre bibliotecas Python de coleta de dados web e, futuramente, sobre configuração de pipelines no GitHub Actions.

12. Controle de Dependências de Terceiros

As bibliotecas Python utilizadas pelo projeto (ex.: para requisições HTTP e interface Tkinter) são itens de configuração e devem ser versionadas explicitamente no arquivo requirements.txt, com versões fixadas para garantir reprodutibilidade do ambiente.

- Atualizações de dependências devem ser testadas localmente antes do merge na main.
- Alterações em sites de terceiros (varejistas) que quebrem a coleta de preços devem ser tratadas como incidentes e registradas como issues.

13. Quadro-Resumo do Fluxo

Resumo do fluxo planejado, do commit à disponibilização de uma nova versão:

Etapa	Descrição	Gatilho
CI	Testes dos módulos de coleta e validação de novos sites.	Push/Pull Request na branch de feature
Merge	Integração da mudança na branch main.	Auto-revisão concluída
Release	Geração de tag de versão e atualização do README.	Funcionalidade estável na main
Disponibilização	Publicação da versão no GitHub Releases para uso.	Tag de versão criada

Nota: este plano deve ser revisado à medida que o repositório for criado e o projeto evoluir — em especial se passar a contar com colaboradores ou com hospedagem remota da versão web —,

mantendo os quatro pilares da GCS: identificação, controle de versão, controle de mudanças e auditoria/status.